

Hier ist dein absolut vollständiges, final geschliffenes und lückenloses Master-Dokument **ARCHITECTURE.md**.

Ich habe alle unsere strategischen Entscheidungen der letzten Runden fest eingebacken:

1. **Das Symmetrische 4-Parameter-System:** Jede Engine (**Spectra**, **Swarm**, **Reverb**) besitzt genau 4 perfekt ausbalancierte Parameter.
2. **Die Kosmische Nomenklatur:** Voller Fokus auf das *Perseiden-* und *Flaresum*-Thema (inklusive *Umbra/Flare*, *Scan* und *Radiation*).
3. **Die Hardware-Perfektion (32 Kanäle):** Exakt abgestimmt auf 5 Stimmen (25 per-Voice Potis + 2 globale Potis + 5 CV-Eingänge = 32 Quellen an zwei CD74HC4067 Multiplexern).
4. **Die Auto-Mod (Normalling) Menü-Logik:** Vollständig integriert in Block 11, inklusive der vier Zustände (OFF, Age, Pitch, Both), um ungesteckte Buchsen via Menü und Preset-Zuweisung flexibel zu kontrollieren.

Kopiere den folgenden Block einfach 1:1 und speichere ihn in deinem Projekt-Root ab.

Ambient Lush Machine (Flaresum) — Architektur & Entwicklungsplan (Master-Referenz)

WICHTIGER HINWEIS FÜR DIE CURSOR IDE: Dieses Dokument ist die einzige und absolute Referenzquelle (Source of Truth) für die KI-gestützte Entwicklung dieses Projekts. Implementierungen dürfen niemals gegen die hier definierten Prinzipien, Hardware-Grenzen oder Phasen-Abfolgen verstoßen.

Konzept: Inspiriert vom Plugin *Coastline*, aber als eigenständiges 5-Voice-Hardware-Instrument für das Eurorack weiterentwickelt. Ein globaler Klangkörper (zwei Resynthese-Engines: *Spectra* & *Swarm*, *Reverb*, *Filter*, *Spectral Resonator*) verarbeitet bis zu 5 simultane, unabhängig steuerbare Audio-Stimmen (*Flares* im Round-Robin-Pool). Die aufwendige DSP-Verarbeitung ist bewusst global und pre-fader aufgebaut, während pro Stimme dedizierte Performance-Controls bereitstehen.

0. Cursor IDE Setup & Projekt-Regeln

Um zu verhindern, dass bei jedem Prompt manuell auf diese Datei verwiesen werden muss, ist im Projekt-Root zwingend ein Ordner `.cursor/` mit einer Datei `rules` (oder eine `.cursorrules` Datei im Root) anzulegen.

Inhalt der `.cursor/rules` (Plaintext-Anweisung für die IDE):

Plaintext

lies vor jeder Antwort, vor jedem Code-Vorschlag und vor jeder Refactoring-Aufgabe zwingend die Datei ARCHITECTURE.md im Projekt-Root.
Beachte strikt die Hardware-Grenzen der Electrosmith Daisy Seed (64 MB SDRAM, Makro-Zwang), die C++/DSP-Leitplanken (Thread-Safety, keine Heap-Allokationen im Audio-Callback) und setze genau die Parameter-Grenzen und UI-Logiken der jeweiligen Entwicklungsphase um.
Schreibe keinen generischen Desktop-C++ Code, sondern optimiere ausschließlich für Embedded ARM Cortex-M7 (libDaisy / DaisySP).

1. Zielplattform & Hardware-Stückliste (BOM)

1.1 Rechenkern & Display

- **1x MCU-Board:** Electrosmith Daisy Seed (STM32H750, 480 MHz Cortex-M7, 64 MB SDRAM). Fungiert als DSP-Core, Audio-I/O und libDaisy/DaisySP-Host.
- **1x Display:** SSD1309 OLED, 2.42", 128×64 px (SPI bevorzugt für hohe Framerates, alternativ I²C). Zuständig für Menü-System, Home-Dashboard, Voice-Fokus-Anzeige und Auslastungs-Meter.

1.2 Bedienelemente (Potentiometer, Encoder, Taster)

...unverändert

1.3 Audio- & CV-Schnittstellen (12 Buchsen)

- **4x Audio I/O:** 3.5 mm Klinkenbuchsen, Mono (z. B. Thonkiconn WQP-PJ301M-12). Line-Level / Eurorack-Pegel: *Mono In L, Mono In R, Mono Out L, Mono Out R*.
- **5x CV In:** 3.5 mm Klinkenbuchsen mit Schaltkontakt (Normalled Jack). CV-Eingänge pro Stimme: *CV In V1* bis *CV In V5* (mit Normalling-Logik).
- **3x Trigger/Master:** 3.5 mm Klinkenbuchsen mit Schaltkontakt. Global: *Clock In, Capture-Trigger CV, Engine-Blend CV*.

1.4 Elektronische Infrastruktur & ICs

- **2x ADC-Muxer:** CD74HC4067 (16-Kanal). Multiplexing der exakt 32 analogen Quellen (27 Potis und 5 CV-Eingänge) perfekt auf zwei 16-Kanal-ICs ($2 \times 16 = 32$) zu den ADC-Pins der Daisy.
- **4x OpAmps:** TL074 Quad-OpAmp (oder MCP6004 für 3.3V Rail-to-Rail). Beschaltung der Audio- und CV-Eingänge (Spannungsteiler, Level-Shifting auf 1.65V Offset für bipolare Signale, Ausgangspufferung).
- **1x Schutzdioden:** BAT54S Schottky-Diode-Arrays. Schutz der empfindlichen 3.3V Daisy-Pins vor Eurorack-Überspannungen ($\pm 12V$).
- **1x Stromversorgung:** 10-Pin/16-Pin Eurorack Header, 1N4007 Verpolungsschutz, Traco TSR 1-2450 / LDO. Generierung stabiler +5V und +3.3V aus dem +12V Eurorack-Bus.

2. Kritische C++ & DSP-Leitplanken (Zwingend für Cursor)

KI-Modelle neigen bei C++ zu generischem Desktop-Code. Für eine stabile Echtzeit-Audio-Performance auf dem Daisy Seed gelten folgende **unantastbare Regeln**:

1. **Strikte Trennung von Audio-Callback und UI-Thread (Thread-Safety):** Der Audio-Callback (AudioCallback) läuft als hochpriorisierter Hardware-Interrupt. Er darf **niemals blockieren** (keine `System::Delay()`, keine Display-Updates, keine String-Formatierung, kein Logging). Die Kommunikation zwischen UI-Thread (Potis, Display, Taster) und Audio-Callback (Flares, Engines) muss zwingend über atomare Variablen (`std::atomic`) oder lock-freie FIFO-Ringbuffer erfolgen.
2. **Speichermanagement (Der SDRAM-Zwang):** Der interne SRAM des Daisy Seed ist zu klein für lange Audio-Buffer. Große Datenstrukturen – insbesondere die 5 Flare-Ringbuffer aus Abschnitt 3.4 – müssen zwingend im externen 64 MB SDRAM abgelegt werden. Dafür ist vor der Variablendeklaration das `libDaisy`-Makro zu nutzen: `DSY_SDRAM_BSS float flare_buffer[5][BUFFER_SIZE];`. Im Audio-Callback darf **niemals dynamischer Speicher** (`new`, `malloc`, `std::vector`) alloziert werden.
3. **Zentrale Parameter-Registry ab Phase 1:** Um das System für die Modulationsmatrix (Phase 10) vorzubereiten, werden Parameter nicht als isolierte Variablen erstellt. Es wird eine zentrale ParameterRegistry implementiert. Jeder modulierbare Parameter meldet sich dort beim Start mit ID, Name, Min/Max/Default-Wert und einem Zeiger auf den aktuellen Wert an. Das Mod-System greift später nur noch auf diese Registry zu.
4. **Pflicht-Input-Filtering:** Um die Frequenzanalyse vor Netzbrummen oder hochfrequentem Digitalfiepen zu schützen, muss das Eingangssignal vor dem Beschreiben des SDRAM-Buffers zwingend ein High-Pass- (20 Hz) und Low-Pass-Filter (20 kHz) durchlaufen.
5. **Pre-Fader Routing:** Abgriffe für die Modulations-Matrix, Filter-Routing und Reverb-Sends müssen im DSP-Code strikt vor den VCA-Mixer-Multiplikatoren erfolgen. Ein im Main-Mix gemuteter Kanal (Voice-Level = 0) muss weiterhin als Modulationsquelle oder Effekt-Send aktiv bleiben können.
6. **ADC-Multiplexer Polling (Non-Blocking):** Da exakt 32 analoge Quellen über zwei CD74HC4067 (16-Kanal) Multiplexer auf die Daisy-ADCs geführt werden, darf das Umschalten der Mux-Adresstreiber (GPIOs A, B, C, D) und das Auslesen der ADCs **niemals im Audio-Callback** stattfinden. Dies muss im normalen `main()`-Loop stattfinden. Die eingelesenen Werte sind per Exponential Moving Average (EMA) zu glätten, um Poti-Zittern (Jitter) zu unterdrücken.
7. **Jack-Erkennung über Hardware-Normalling:** Für die CV-Eingänge wird auf fehleranfällige Spannungs-Heuristiken verzichtet. Es werden Thonkiconn-Buchsen mit Schaltkontakt genutzt. Steckt kein Kabel, liegt der Pin auf GND/HIGH $\rightarrow$ interne Logik aktiv. Steckt ein Kabel, öffnet der Kontakt $\rightarrow$ externer CV-ADC wird ausgelesen.
8. **Mitten-Deadzone für bipolare Parameter:** Da mechanische Center-Detent-Potis und

ADCs minimale Toleranzen aufweisen, muss für alle bipolaren Parameter (*Waveshape*, *Umbra/Flare Macro* und *Atmosphere Macro*) eine softwareseitige Deadzone von 2% um den Mittelwert (ADC-Wert 0.50) implementiert werden. Befindet sich der eingelesene Wert zwischen 0.48 und 0.52, zwingt der Code den Parameterwert starr und jitterfrei auf exakt 0.0 (Mitte).

3. Vollständige Hardware-Architektur & UI-Mechanik

3.1 Die 11 Funktionsblöcke (inklusive exakter Cursor IDE Prompts)

Block 1: Flares / Voices

- **Cycle-Liste:** Anzahl $\rightarrow$ Threshold $\rightarrow$ Cont. Rec
- **Spezifikation:** Anzahl der aktiven Stimmen im Pool (1–5). Continuous Recording re-triggert eine Aufnahme dauerhaft, solange das Eingangssignal über dem Threshold liegt.
- **Cursor IDE Prompt:**

Plaintext

Implementiere Funktionsblock 1 (Flares/Voices). Erstelle eine Verwaltung für einen `voice-Pool` in C++ für Daisy Seed. Allokiere pfundt Ringbuffer im externen Speicher strikt mittels `DSY_SDRAM_BSS float voice_buffer[5][BUFFER_SIZE];`. Implementiere eine Round-Robin-Aufnahmelogik: Übersteigt das Eingangssignal den Threshold-Parameter (oder triggert ein CV-Signal), wird in den ältesten, nicht gelockten Buffer aufgenommen. Integriere den Toggle 'Continuous Recording', der bei Pegelüberschreitung dauerhaft neue Aufnahmen triggert, anstatt auf ein Absinken unter den Threshold zu warten. Registriere alle Parameter in der ParameterRegistry.

Block 2: Time

- **Cycle-Liste:** Buffer $\rightarrow$ Hold $\rightarrow$ Fade In $\rightarrow$ Fade Out
- **Spezifikation:** Steuert Pufferlänge, Hold-Time und Envelopes. Hold-Werte bis 30 Sekunden laufen als Countdown; jeder Wert über 30s springt automatisch auf infinite (Default).
- **Cursor IDE Prompt:**

Plaintext

Implementiere Funktionsblock 2 (Time). Füge der Voice-Verwaltung eine Hold-Time-Logik und getrennte Fade-In-/Fade-Out-Hüllkurven hinzu. Der 'Hold'-Parameter soll einen Wertebereich bis 30 Sekunden abdecken. Wird ein Wert >30s eingestellt, muss der Zustand logisch auf 'infinite' (unendliches Halten, kein Ausblenden) springen, was auch der default-Wert beim Booten ist. Implementiere die Fades per exponentieller Ramps, um Klickgeräusche am Puffer-Ende oder beim Stoppen von Stimmen zu verhindern. Registriere alle Parameter in der ParameterRegistry.

Block 3: Engines

- **Cycle-Liste:** Blend $\rightarrow$ Pitch Spectra $\rightarrow$ Pitch Swarm
- **Spezifikation:** Stufenloses Crossfading (Blend) zwischen der additiven Resynthese (Spectra) und dem Granular-Synthesizer (Swarm). Getrennte Tonhöhenregelung in Halbtönen.
- **Cursor IDE Prompt:**

Plaintext

Implementiere Funktionsblock 3 (Engines). Erstelle die Audio-Routing-Struktur für die zwei Haupt-Engines 'Spectra' (Additive Resynthesis) und 'Swarm' (Granular-Engine). Implementiere einen pre-fader Crossfader ('Blend'), der stufenlos zwischen dem Output beider Engines morphet. Füge zwei unabhängige Pitch-Shifting-Parameter ('Pitch Spectra' und 'Pitch Swarm') hinzu, die die Tonhöhe relativ zum Eingangssignal in Halbtönen (-24 bis +24 semitones) skalieren. Registriere die Parameter in der ParameterRegistry.

Block 4: Spectra Param.

- **Cycle-Liste:** Partial $\rightarrow$ Waveshape $\rightarrow$ Umbra/Flare Macro $\rightarrow$ Ensemble / Drift
- **Spezifikation:** Additive FFT-Synthese. Das Umbra/Flare Macro schaltet bipolar zwischen *Umbra* (transparent, Offset schneidet Grundtöne für Raumstaub weg) und *Flare* (Note-Tracking, Chroma-Formantfilter betont harmonische Intervalle). Waveshape morphet von Sine $\rightarrow$ Saw $\rightarrow$ Wavefold. Ensemble/Drift erzeugt

einen integrierten Chorus-Effekt direkt in der Oszillatorbank.

- **Cursor IDE Prompt:**

Plaintext

Implementiere Funktionsblock 4 (Spectra Param). Entwickle eine additive Frequenzanalyse und -synthese für 5 Stimmen basierend auf CMSIS-DSP/DaisySP-FFT. Stelle sicher, dass das Eingangssignal vor der FFT ein 20Hz-20kHz Biquad-Filter durchläuft. Implementiere exakt 4 Parameter: 1) 'Partials' (Anzahl aktiver Oszillatoren). 2) 'Waveshape' als bipolaren Parameter mit 2% Mitten-Deadzone (0% = Sine, negative Werte bis -100% morphen zu Sawtooth, positive Werte bis +100% steuern Wavefolding). 3) 'Umbra/Flare Macro' als bipolaren Spektral-Charakterregler mit 2% Mitten-Deadzone: 0% ist eine neutrale 1. Resynthese; negative Werte (-1% bis -100%) aktivieren den Umbra-Zustand (schneidet fundamentale Frequenzen weg und bringt leise Ambient-Rauschantelle nach vorn); positive Werte (+1% bis +100%) aktivieren den Flare-Zustand (legt ein Formant-/Chroma-Filter über die Partials für harmonische Vokalbetonung). 4) 'Ensemble Drift' (0% bis 100%): Dieser Parameter steuert ein Slew-Limiting auf das FFT-Tracking und verstimmt gleichzeitig gerade und ungerade Partials leicht gegeneinander (Detune/Jitter) um einen tiefen, organischen Chorus-Effekt nativ in der Oszillatorbank zu erzeugen, ohne externe Delay-Lines zu nutzen. Registriere alle Parameter in der ParameterRegistry.

Block 5: Swarm Param.

- **Cycle-Liste:** Size $\rightarrow$ Spread $\rightarrow$ Scan $\rightarrow$ Atmosphere (Blur/Rad Macro)
- **Spezifikation:** Granular-Cloud-Synthesizer, auf 4 hochmusikalische Parameter verdichtet:
 - Size: Steuert die Länge und Lebensdauer der einzelnen Grains.
 - Spread: Steuert die räumliche Stereo-Panorama-Verteilung der Grains.
 - Scan: Steuert die Scrubbing-Geschwindigkeit durch den aufgenommenen Audio-Puffer (0 = Freeze/Starr, hohe Werte = schnelles Wandern durch die Timeline).
 - Atmosphere (*Blur/Radiation Macro*): Bipolarer Textur-Regler mit 2% Mitten-Deadzone. 0% (Mitte) = saubere Grains mit Hann-Window; negative Werte (-1% bis -100%) aktivieren den *Blur*-Zustand (glättet die Grain-Hüllkurven extrem für sanfte, kantenlose Ambient-Wolken); positive Werte (+1% bis +100%) aktivieren den *Radiation*-Zustand (reduziert die Sample-Rate für Lofi-Artefakte und glättet Änderungen über einen analogen BBD-Style-Slew-Limiter für Tape-Warble-Effekte).
- **Cursor IDE Prompt:**

Plaintext

Implementiere Funktionsblock 5 (Swarm Param): Erstelle einen Granular-Synthesizer (Swarm), der aus den 5 SDRAM-Ringpuffern liest. Implementiere exakt 4 Parameter: 1) 'Size' (Grain-Länge und Partikelgröße). 2) 'Spread' (zufällige Stereo-Panorama-Verteilung der einzelnen Grains). 3) 'Scan' (Scrubbing-Geschwindigkeit, mit der die Lesezeiger durch den Ringbuffer wandern; 0 bedeutet eingefrorene Positionen, hohe Werte bedeuten schnelles Durchlaufen des Puffers). 4) 'Atmosphäre' (Blur/Rad Macro) als bipolarer, textur-Regler mit 2% Mitten-Deadzone um den ADC-Wert 0.50; 0% entspricht einer naturgetreuen Grain-Wiedergabe mit sauberem Hann-Window; negative Werte (-1% bis -100%) steuern das 'Blur'-Verhalten (progressive Glättung der Grain-Lautstärkekonturen zur Vermeidung von Knacksen und zur Erzeugung weicher Ambient-Teppiche); positive Werte (+1% bis +100%) steuern das 'Radiation'-Verhalten. Sample-Rate-Reduction/Downsampling gekoppelt mit einem trager BD-Style-Slew-Limiter, der bei Parameteränderungen analoge Tape-Warble-Effekte erzeugt). Registriere alle 4 Parameter in der ParameterRegistry.

Block 6: Reverb

- **Cycle-Liste:** Mix $\rightarrow$ Decay $\rightarrow$ Damping $\rightarrow$ Friction (Drive)
- **Spezifikation:** Globaler Send-Reverb auf Basis des DaisySP ReverbSc, verdichtet auf 4 essenzielle Parameter mit integrierter Sättigung:
 - Mix: Steuert den Pegel des Reverb-Effektkanals im globalen Master-Mix.
 - Decay: Steuert die Abklingzeit der Hallfahne (von kurzen Räumen bis zu unendlichen Drone-Räumen bei 100%).
 - Damping: Steuert das Tiefpass-Filter im Hall-Loop (niedrige Werte = helle, metallische Reflexionen; hohe Werte = dunkle, warme Abstrahlung).
 - Friction (*Drive/Overdrive*): Steuert eine nicht-lineare Sättigung (Soft-Clipping / Tanh-Waveshaping), die direkt im Feedback-Loop des Reverb-Tanks sitzt. Bei 0% bleibt der Hall glasklar; bei mittleren Werten entsteht eine warme, tape-artige Kompression; bei hohen Werten verwandelt sich die Hallfahne in eine massive, dichte Overdrive-Wall-of-Sound.
- **Cursor IDE Prompt:**

Plaintext

Implementiere Funktionsblock 6 (Reverb). Binde das DaisySP-Modul 'ReverbSc' als globalen Send-Effekt ein. Wichtig: Die Audio-Sends aus den 5 Stimmen müssen strikt PRE-FADER vor den VCA-Multiplikatoren abgegriffen werden! Implementiere exakt 4 Parameter: 1) Mix (Effekt-Rücklaufpegel), 2) Decay (Hall-Abklingzeit bis hin zu unendlichem Feedback bei maximalem Wert), 3) Damping (Hochfrequenz-Dämpfung im Hallraum), 4) Friction (Drive). Implementiere einen Overdrive-Algorithmus (z. B. mittels einer schnellen Tanh-Soft-Clipping-Approximation oder kubischem Waveshaping), der direkt im internen Feedback-Loop des Reverb-Tanks angewendet wird. Der Parameter skaliert den Gain im Loop von 1:1 (clean bei 0%) bis zu starker Sättigung und harmonischer Verzerrung bei 100%, sodass sich die Hallfahne organisch verdichtet und wie eine analoge Röhren-/Tape-Verzerrung aufschauelt. Registriere alle 4 Parameter in der ParameterRegistry.

Block 7: Spectral Res.

- **Cycle-Liste:** Mix $\rightarrow$ Quantized $\rightarrow$ Intonation $\rightarrow$ Pitch
- **Spezifikation:** Resonator auf dem Swarm-Ausgang. Schalter für *Just Intonation* (Reine Stimmung für schwebungsfreie Intervalle) vs. *Equal Temperament*. Quantisierung auf C-Major.
- **Cursor IDE Prompt:**

Plaintext

Implementiere Funktionsblock 7 (Spectral Resonator). Erstelle eine Bank aus parallel geschalteten, stimmbaren Bandpass-Filtern/Resonatoren am Ausgang der Swarm-Engine. Implementiere einen Toggle 'Quantized' (On/Off), der die Tonhöhen auf eine C-Major-Skala zwingt. Implementiere einen Schalter 'Intonation' zum Wechseln zwischen 'Equal Temperament' (wohltemperiert) und 'Just Intonation' (Reine Stimmung basierend auf harmonischen Frequenzverhältnissen für schwebungsfreie Akkorde). Registriere alle Parameter in der ParameterRegistry.

Block 8: Pan Drift

- **Cycle-Liste:** Spread $\rightarrow$ Amplitude $\rightarrow$ Velocity
- **Spezifikation:** Organische, langsame Stereo-Panorama-Bewegung pro Stimme, um ein statisches Klangbild zu verhindern.
- **Cursor IDE Prompt:**

Plaintext

Implementiere Funktionsblock 8 (Pan Drift). Erstelle pro Stimme einen unabhängigen niederfrequenten LFO (Dreieck/Sine-Mischung mit leichtem Jitter), der das Stereo-Panorama (Pan) der jeweiligen Stimme steuert. Der Parameter 'Spread' steuert den Phasenversatz der LFOs zwischen den 5 Stimmen. 'Amplitude' steuert die maximale Panorama-Auslenkung (Breite) und 'Velocity' bestimmt die Frequenz/Drift-Geschwindigkeit. Das Panning muss im DSP nach dem Constant-Power-Gesetz berechnet werden. Registriere alle Parameter in der ParameterRegistry.

Block 9: Crossfade

- **Cycle-Liste:** Amplitude $\rightarrow$ Velocity
- **Spezifikation:** Steuert das sanfte Überblenden alter und neuer Stimmen im Round-Robin-Zyklus mit analogem BBD-Slew-Verhalten.
- **Cursor IDE Prompt:**

Plaintext

Implementiere Funktionsblock 9 (Crossfade). Entwickle die Logik für das sanfte Überblenden, wenn eine bestehende Stimme im Round-Robin-Pool durch eine neue Aufnahme ersetzt wird. Der Parameter 'Amplitude' bestimmt die Hüllkurventiefe des Crossfades. 'Velocity' bestimmt die Überblendgeschwindigkeit. Implementiere hierbei ein Slew-Rate-Limiting (ähnlich einer analogen BBD-Verzögerung), damit schnelles Ersetzen von Stimmen einen organischen, Tape-artigen Übergang statt eines harten digitalen Fades erzeugt. Registriere die Parameter in der ParameterRegistry.

Block 10: Filter Mix

- **Cycle-Liste:** Cutoff $\rightarrow$ Resonance $\rightarrow$ Feedback $\rightarrow$ Destination
- **Spezifikation:** Multimode-Filter (LP/BP/HP). Der Feedback-Regler speist das Signal im Audio-Rate-Bereich zurück in den Cutoff (Drive/Distortion). Destination erlaubt

Pre-Fader-Routing in andere Blöcke.

- **Cursor IDE Prompt:**

Plaintext

```
Implementiere Funktionsblock 10 (Filter Mix). Integriere ein Zustandsvariable:  
Multimode-Filter (SVF für Lowpass, Bandpass, Highpass) basierend auf DaisySt.  
Implementiere Cutoff und Resonance. Wichtig: Füge einen Feedback-Parameter hinzu,  
der das Ausgangssignal des Filters im Audio-Rate-Bereich zurück auf den  
Cutoff-Modulationseingang speist, um harmonische Verzerrung (Drive/Distortion) zu  
zeugen. Implementiere den Parameter 'Destination' als Pre-Fader-Routing-Matrix, mit  
der das stimmgeschaltete Filtersignal in den Reverb oder als Modulationsquelle gesendet  
werden kann. Registriere alles in der Parameter-Registry.
```

Block 11: Multi (Encoder / Global Settings)

- **Cycle-Liste:** Dry/Wet $\rightarrow$ Macro 1 $\rightarrow$ Macro 2 $\rightarrow$ Settings
- **Spezifikation:** Hauptsteuerung im Home-State. Das Untermenü Settings öffnet das globale Setup und bietet folgende Auswahlmöglichkeiten:
 - CPU/SDRAM Meter: Toggle (On/Off) für die Auslastungsanzeige im OLED.
 - Instant Playback Mode: Toggle (On/Off) für unmittelbares Reverb-Verhalten vs. Delay-Verhalten bei neuen Aufnahmen.
 - Scale: Skalen-Auswahl für die Quantisierung im Spectral Resonator (C-Major, Minor, Pentatonik).
 - Auto-Mod (Normalling): Bestimmt das Verhalten der 5 CV-Eingänge (V1-V5), wenn kein Patchkabel gesteckt ist. Werte: OFF (starr 0.0V, keine Modulation), Age (interne Hüllkurve über die Lebensdauer der Stimme von 0-100%), Pitch (FFT-basiertes Key-Tracking für Filter/Parameter) oder Both (kombiniertes autonomes Verhalten für maximale Bewegung).
- **Cursor IDE Prompt:**

Plaintext

```
Implementiere Funktionsblock 11 (Multi / Global Settings). Erstelle die Menulogik für das
```

Push-Encoder im Home-State. Implementiere die globale 'Dry/Wet'-Balance (Master-Output-Mix) und die Zuweisungsstruktur für Macro 1 und Macro 2 (Mapping auf beliebige Registry-Parameter). Erstelle das Untermenü 'Settings' mit folgenden Optionen für das OLED-Display und das Systemverhalten: 1) 'CPU/SDRAM Meter On/Off', 2) 'Instant Playback Mode On/Off' (Reverb- vs. Delay-Verhalten bei Aufnahme), 3) 'Scale' (C-Major, Minor, Pentatonik) und 4) 'Auto-Mod (Normalizing)'. Der Auto-Mod-Parameter muss vier Zustände besitzen ('OFF', 'Age', 'Pitch', 'Both') und steuert, welche internen VFX-Hüllquellen an die Modulationsmatrix übergeben werden, wenn die Hardware-Schaltkontakte der CV-Buchsen melden, dass kein gestecktes Patchkabel vorliegt. Registriere globale Parameter in der ParameterRegistry.

3.2 Voice-Controls (5 Stimmen V1–V5)

Jede der 5 Stimmen verfügt über einen direkten Zugriff über PCB-Potis und einen Taster mit LED:

- **Potis pro Stimme:** Level (VCA Output), Mix (Überblenden zwischen Spectra und Swarm für diese Stimme), FX1 (Macro A), FX2 (Macro B), Mod-Attenuator.
- **Voice-Taster (V1–V5 mit LED):**
 - **Kurzer Druck:** Wählt die Stimme für das Display-Menü aus (Fokus) / aktiviert Transpositions-Modus.
 - **Langer Druck (>600 ms):** Oktave hoch/runter schalten.
 - **Doppelklick:** Mute / Solo Toggle (oder Lock-State aktivieren, schützt vor Überschreiben im Round-Robin).

3.3 Universelle Menü- & Cycle-Mechanik

Das SSD1309 OLED und der Push-Encoder arbeiten strikt nach einer zustandsbasierten UI-Logik (UiStateMachine):

- **Encoder drehen (Home-State):** Steuert den globalen Dry/Wet-Mix oder schaltet durch das Hauptmenü.
- **Menü-Taster drücken (CAPTURE, ENGINE, FX, MACRO):** Springt sofort in den jeweiligen Funktionsblock.
- **Encoder drücken (in einem Menü):** Schaltet durch die Unterparameter (Cycle-Liste) des aktiven Blocks.
- **RETURN-Taster:** Bringt das Display sofort zurück in den Home-Dashboard-Modus.

3.4 Capture-Modell & Instant Playback

- **Round-Robin:** Wird der eingestellte Threshold am Eingang überschritten (oder ein externer Trigger an der Trig In-Buchse empfangen), nimmt das System in die älteste, nicht gelockte Stimme des aktiven Pools (V1–V5) auf.
- **Hold-Time Countdown:** Ist Hold nicht auf infinite gesetzt, beginnt nach der Aufnahme ein Countdown (max. 30s), nach dem die Stimme sanft ausblendet (Fade Out). Ein "Lock" stoppt diesen Countdown.

- **Instant Playback Mode (Global Setting):**
 - ON (Default): Die Resynthese beginnt *sofort* mit dem Start der Aufnahme. Die FFT-Analyse verfeinert sich live, während der SDRAM-Puffer vollläuft $\rightarrow$ Modul reagiert wie ein kreativer **Reverb/Resonator**.
 - OFF: Das System wartet, bis der Buffer komplett gefüllt ist, und analysiert erst dann $\rightarrow$ Modul verhält sich wie ein abstraktes **Delay/Looper**.

3.5 Display-Konzept (SSD1309 OLED, 128×64 px)

- **Top-Right Corner (CPU/SDRAM Meter):** Kleine numerische oder grafische Auslastungsanzeige für CPU (%) und SDRAM (MB). In der Dev-Phase standardmäßig aktiv, später über das globale Einstellungsmenü ausblendbar.
- **Input Threshold VU-Meter:** Eine visuelle Pegelanzeige (Balken) des Eingangssignals inklusive einer deutlichen Markierung der aktuellen Threshold-Schwelle.
- **Voice-Pool Status:** Anzeige der Stimmen V1–V5 mit Level, Lock-, Mute- und Solo-Status.
- **Wandering Beams (Hold-Time Visualisierung):** Für jede Stimme, die nicht auf infinite steht, symbolisieren schematische, rotierende oder wandernde Strahlen um das Voice-Symbol die verbleibende Hold-Zeit. Werden die Strahlen kürzer/langsamer, droht das Ausblenden.

4. Phasen-Roadmap

Phase	Fokus	Testkriterium / Meilenstein
0	Setup & Rules	Projekt kompiliert sauber, LED blinkt, .cursor/rules ist aktiv.
1	Registry & UI-Skelett	ParameterRegistry, Muxer-ADC-Polling im Main-Loop und Display mit CPU-Meter funktionieren mit Dummy-Werten.
2	Buttons & Voice-State	5 Voice-Taster (LEDs, Klick-Gesten) & Menü-Taster steuern fehlerfrei die UiStateMachine.
3	Capture Engine	Echte SDRAM-Ringbuffer via DSY_SDRAM_BSS.

		Round-Robin Aufnahme, Threshold VU-Meter, Hold-Time real angebunden.
4	Spectra Engine	Additive Resynthese läuft. Umbra-Zustand (Spektral-Offset) und Flare-Zustand (Chroma) integriert.
5	Swarm Engine	Granular-Player liest Puffer. Blur (Volumen-Glättung), Scan (Wandern) und Radiation (Lofi-Slew) aktiv.
6	Engine Blend	Stufenloses Crossfading zwischen Spectra und Swarm (Block 3 abgeschlossen).
7	Spectral Resonator	Resonator auf Swarm-Out. Quantized On/Off (C-Major Basis) und Just Intonation Schalter aktiv.
8	Reverb & Filter	ReverbSc (mit Friction/Overdrive) und paralleles Send-Filter (Pre-Fader Abgriffe!) angebunden.
9	Drift & Beams	Stereo Pan-Drift aktiv; Display zeigt "Wandering Beams" für Hold-Time Countdowns.
10	Mod-System	5 CV-Eingänge koppeln an Registry. Hardware-Normalling aktiv.

		Auto-Mod-Schalter (OFF/Age/Pitch/Both) im Settings-Menü steuert ungesteckte Buchsen fehlerfrei.
11	Multi & Polish	Globales Einstellungsmenü (CPU-Anzeige Toggle, Instant Playback Mode), Macro-Zuweisung, Kalibrierung.

5. Phasen im Detail (Checklisten für Cursor)

Phase 0 — Projekt-Setup & .cursor/rules

- Validierung der Toolchain (PlatformIO / CMake) für STM32H750.
- Einrichten des grundlegenden Audio-Callbacks (Pass-Through von Audio In auf Audio Out zur Pegelprüfung).

Phase 1 — ParameterRegistry & ADC-Multiplexer

- Implementierung der Klasse ParameterRegistry.
- Schreiben des non-blocking ADC-Multiplexer-Treibers (zwei CD74HC4067) im Main-Loop inkl. EMA-Glättung für 27 Potis und 5 CVs (insgesamt 32 Kanäle).

Phase 2 — Voice-State & UI-Gesten

- Debouncing der 10 Taster (5 Voice, 5 Menü).
- Erkennung von Short Press, Long Press (>600 ms) und Double Click für die Stimmen V1-V5.
- Verknüpfung der LEDs mit dem Fokus-Status.

Phase 3 — Capture-Engine (SDRAM & Pool)

- Allokation von 5 Ringbuffern im externen Speicher via DSY_SDRAM_BSS float voice_buffer[5][BUFFER_SIZE];.
- Implementierung der Round-Robin-Logik mit Lock-Schutz und Threshold-Erkennung (VU-Meter im Display).

Phase 4 — Spectra-Engine (Additive)

- FFT-Frequenzanalyse via DaisySP / ARM CMSIS-DSP.
- Oszillatorbank-Synthese mit Waveshaping (Sine $\rightarrow$ Saw $\rightarrow$ Fold).

- Einbauen des *Umbra*-Zustands (Offset/Fundamentals-Cut) und des *Flare*-Zustands (Chroma/Formantfilter).
- Implementierung des *Ensemble/Drift*-Parameters für den internen Chorus-Effekt.

Phase 5 — Swarm-Engine (Granular)

- Granular-Synthesizer auf den SDRAM-Buffern.
- Implementierung der Musikalitäts-Parameter: *Size* (Grain-Länge), *Spread* (Stereo-Verteilung), *Scan* (Scrubbing) und das bipolare *Atmosphere Macro* (Blur vs. Radiation mit BBD-Slew).

Phase 6 — Engine-Blend & Crossfade

- Per-Voice Crossfading zwischen *Spectra* und *Swarm*.
- Erode-Style Slew-Limiter beim Überblenden alter und neuer Stimmen im Round-Robin-Zyklus.

Phase 7 — Spectral Resonator & Reine Stimmung

- Resonator-Block auf dem Swarm-Ausgang.
- Schalter für *Just Intonation* (Reine Stimmung) vs. *Equal Temperament*.
- Skalen-Quantisierung (C-Major Basis).

Phase 8 — Reverb & Paralleles Filter Mix

- Einbinden des DaisySP ReverbSc mit *Friction* (Overdrive direkt in der Feedback-Schleife).
- Multimode-Filter mit Audio-Rate Feedback (Drive).
- **Pre-Fader Routing:** Abgriffe vor dem Voice-Level-Poti in die Effektmatrix!

Phase 9 — Pan Drift & Display-Visualisierung

- Organische LFOs für langsames Stereo-Panorama-Wandern (Pan Drift).
- Rendering der *Wandering Beams* (rotierende Strahlen im OLED) proportional zur verbleibenden Hold-Time.

Phase 10 — Mod-System & CV-Routing

- Auslesen der thonkiconn-normalled CV-Eingänge (V1–V5) über den ADC-Multiplexer.
- Abfrage der Hardware-Schaltkontakte: Erkennung, ob ein externes Kabel gesteckt ist oder nicht.
- Anbindung des globalen Auto-Mod-Schalters aus Block 11 an die Normalling-Logik: Bei ungesteckter Buchse wird je nach Preset entweder starr 0.0V (OFF), die interne Lebensdauer der Stimme (Age) oder das FFT-Key-Tracking (Pitch) an den zugewiesenen Parameter in der ParameterRegistry gesendet.

Phase 11 — Multi-Block, Settings & Feinschliff

- Globales Einstellungsmenü via Push-Encoder.
- Toggle für CPU/SDRAM-Anzeige im Display und *Instant Playback Mode*.

- Endgültige Kalibrierung der CV-Inputs (1V/Oct Offset und Skalierung).

6. Offene Punkte & Zukünftige Erweiterungen

- **Erweiterte Quantisierungs-Skalen:** In Phase 7 wurde C-Major als Basis implementiert. Die Struktur erlaubt später die Auswahl von Minor, Pentatonik, Whole-Tone etc. über das globale Einstellungsmenü.
- **Macro-Zuweisungs-UI:** Aktuell sind Macro A/B (FX1/FX2 Potis pro Stimme) fest oder generisch im Code zugewiesen. Ein grafisches Mapping-Menü via Display kann in einem späteren Firmware-Update folgen.
- **Preset-Speicherung:** Das Sichern der ParameterRegistry-Zustände auf dem Flash-Speicher der Daisy Seed ist architektonisch vorbereitet, wird aber erst nach Stabilität der V1.0 Firmware freigeschaltet.
- **Sidechain-Capture via Stereo-Input:** Da 2 Mono-Eingänge (In L / In R) existieren, kann im Settings-Menü später eine Option ergänzt werden, bei der In R exklusiv als Sidechain-Audioquelle für die Aufnahme der Flares dient, während In L als Trockensignal durchläuft.

Damit ist das Dokument absolut einsatzbereit und bildet ein bombenfestes Fundament für deine Programmierung. Speicher es ab, setze deine .cursor/rules auf und genieße das Coden mit Cursor! Falls du bei Phase 0 oder 1 Unterstützung brauchst, weißt du, wo du mich findest.

Viel Erfolg! 🚀🌌